Parallelization of an ACO Solver for the Constrained Vehicle Routing Problem

Francesco Zompanti (2012601)

Abstract

I present sequential, OpenMP–MPI, and CUDA implementations of an Ant Colony Optimization solver for the Constrained Vehicle Routing Problem. The final campaign uses three repetitions per point and validates every saved route. On 100 fixed epochs at $n = 32{,}000$, OpenMP reaches 2.76× speedup at 32 threads, whereas the measured four-node MPI configuration is slower than its 32-worker baseline. The complete weak curve retains 87% efficiency at 128 workers relative to one. CUDA reaches $n = 64{,}000$, but unequal search budgets prevent a cross-backend speedup claim. The limited strong-scaling efficiency and inconsistent printed costs in seven multi-rank outputs are reported explicitly.

1 Introduction

I study the **Constrained Vehicle Routing Problem** (CVRP) solved through **Ant Colony Optimization** (ACO), focusing on the implementation choices needed for shared-memory, distributed-memory, and GPU execution.

1.1 Problem formulation

We consider a constrained vehicle routing problem on a complete graph $G = (V, E)$, where $V = \{0, 1, \ldots, n\}$ is the set of nodes and node 0 denotes the depot. The set of customers is therefore $C = \{1, \ldots, n\}$. Each arc $(i, j) \in E$ is associated with a travel cost $c_{ij} \geq 0$, derived in my instances from the Euclidean distance between nodes. A fleet of at most K vehicles is available, and each vehicle has capacity Q. In the instances handled by my solver, customer demands are unitary, that is, $q_i = 1$ for every $i \in C$, while $q_0 = 0$.

A feasible solution is a set of depot-to-depot routes such that each customer is visited exactly once and the total demand served by each route does not exceed the vehicle capacity. Let x_{ij}^k be a binary decision variable equal to 1 if vehicle k traverses arc (i, j) and 0 otherwise. A compact mathematical formulation can be written as follows:

$$\min \sum_{k=1}^{K} \sum_{i \in V} \sum_{j \in V,\, j \neq i} c_{ij} x_{ij}^k$$

subject to

$$\sum_{k=1}^{K} \sum_{j \in V,\, j \neq i} x_{ij}^k = 1 \qquad \forall i \in C$$

$$\sum_{j \in C} x_{0j}^k \leq 1, \qquad \sum_{i \in C} x_{i0}^k \leq 1 \qquad \forall k = 1, \ldots, K$$

$$\sum_{j \in V,\, j \neq i} x_{ij}^k - \sum_{j \in V,\, j \neq i} x_{ji}^k = 0 \qquad \forall i \in V,\ \forall k = 1, \ldots, K$$

$$\sum_{i \in C} q_i \sum_{j \in V,\, j \neq i} x_{ij}^k \leq Q \qquad \forall k = 1, \ldots, K$$

with

$$x_{ij}^k \in \{0, 1\} \qquad \forall i, j \in V,\ i \neq j,\ \forall k = 1, \ldots, K.$$

The objective minimizes the total travel cost of all routes. The first constraint ensures that every customer is served exactly once. The second enforces that each vehicle can leave from and return to the depot at most once. The third is the standard flow-conservation constraint, while the fourth encodes the vehicle-capacity limit. In the specific setting of my project, since all customer demands are equal to one, the capacity constraint can also be interpreted as a bound on the maximum number of customers assigned to each route.

1.2 Sequential baseline analysis

The sequential baseline follows a standard ACO structure with two practical additions: bounded pheromone updates and a repair-oriented route construction phase that preserves feasibility when some customers remain unassigned after the first pass.

Algorithm 1 Sequential ACO baseline

Input: n, K, Q, m, c, $(\alpha, \beta, \rho, \tau_0)$, seed
initialize pheromone matrix τ and heuristic data
if $m \le 0$ **then**
    $m \leftarrow \textsc{ChooseAutoTotalAnts}(n)$
end if
$S^\star \leftarrow \varnothing$; $f^\star \leftarrow +\infty$
$\tau_{\max} \leftarrow \tau_0$; $\tau_{\min} \leftarrow 0.05\tau_0$
for $t \leftarrow 0, 1, 2, \ldots$ until timeout or stagnation **do**
    update candidate scores from τ
    $S^{\text{iter}} \leftarrow \varnothing$; $f^{\text{iter}} \leftarrow +\infty$
    for $a \leftarrow 1$ to m **do**
        build and evaluate one ant solution
        if $f_a < f^{\text{iter}}$ **then**
            $S^{\text{iter}} \leftarrow S_a$; $f^{\text{iter}} \leftarrow f_a$
        end if
    end for
    if f^{iter} improves $f^\star$ **then**
        update $S^\star$, $f^\star$, and pheromone bounds
        reset stagnation counters
    else
        increment stagnation counters
    end if
    evaporate pheromone on all arcs
    reinforce pheromone with S^{iter} and $S^\star$
    clamp τ to $[\tau_{\min}, \tau_{\max}]$
    if short-term stagnation is reached **then**
        mix τ with the initial level τ_0
    end if
end for
return $S^\star, f^\star$

Algorithm 2 Simplified feasible ant construction

function $\textsc{BuildAntSolution}(K, Q, c, \tau)$
    initialize empty solution, visited set, and remaining customers $r \leftarrow n$
    for $k \leftarrow 1$ to K **do**
        start route R_k at depot 0 with load $\ell_k \leftarrow 0$
        while $r > (K - k)Q$ and $\ell_k < Q$ **do**
            choose the next customer by the ACO rule
            if the choice is invalid or already visited **then**
                replace it with the nearest unvisited customer
            end if
            if no feasible customer exists **then**
                break
            end if
            append the customer to R_k, mark it visited, and decrement r
        end while
        close R_k by returning to depot 0
    end for
    if $r > 0$ **then**
        reopen routes from K down to 1
        greedily insert nearest unvisited customers while capacity remains
        close the modified routes again at the depot
    end if
    return the constructed solution
end function

In implementation terms, this pseudo-code is realized through four main components. The instance parser reads TSPLIB-style EUC_2D CVRP inputs, extracts the number of vehicles and the route capacity, validates the unit-demand assumption, and builds the complete distance matrix. The matrix and solution modules provide the aligned dense storage and the route-based solution representation. The ACO support routines manage random-number generation, candidate scoring, candidate selection, and route-construction data. Finally, the optimization loop coordinates ant generation, solution evaluation, best-solution tracking, pheromone bounding, and stagnation recovery.

This structured description is especially useful for the rest of the report, because it isolates the computational kernels that dominate execution time and therefore become the natural targets for optimization and parallelization: ant solution construction, probabilistic next-customer selection, route-cost evaluation, and pheromone management.

1.3 Problem and algorithmic challenges

The project combines combinatorial optimization and constrained route construction, which introduces several challenges:

- the search space grows rapidly with the number of customers;
- the constructive phase must preserve feasibility while still exploring competitive solutions;
- the algorithm repeatedly evaluates many probabilistic choices and route costs, making performance strongly dependent on memory access patterns and implementation efficiency.

2 OpenMP–MPI Implementation

The CPU-parallel backend follows the natural coarse-grained decomposition of ACO. OpenMP threads build independent ant solutions inside each rank, while MPI combines information across ranks. This mapping keeps the sequential decision chain of an individual ant local and exposes parallelism across ants.

2.1 Shared-memory organization

The current implementation creates one persistent OpenMP region around the epoch loop. Thread-private route-construction workspaces avoid sharing the visited set and random-number state. Candidate indices and heuristic scores are stored contiguously, while the distance and pheromone matrices use single-precision aligned storage within the parallel solver. Per-epoch work consists of four main phases: candidate-score refresh, independent ant construction, selection of the best solution, and pheromone evaporation/deposit.

The ant loop uses a runtime-selectable OpenMP schedule. Pheromone evaporation is partitioned statically over the dense matrix; deposits use atomic additions because different routes can touch the same edge. This removes a serial deposit phase at the price of possible atomic contention. The fixed-size strong-scaling campaign in Section 4 is designed to measure the net effect rather than attributing time to any one phase without profiling.

2.2 Distributed-memory synchronization

Each MPI rank owns a replica of the pheromone matrix and receives a disjoint portion of the total ant population. Modified edges are represented as sparse *delta* records containing a matrix index and an increment. The implementation starts a nonblocking `MPI_Iallgatherv`; deltas from the previous epoch are applied before the next local update. This avoids transmitting the complete dense matrix when only route edges have changed.

The sparse exchange does not remove all distributed overhead. Every rank still initializes and evaporates a dense matrix, performs collective coordination, and stores its local replica. Consequently, increasing the number of ranks may expose replicated work even when the communicated payload is sparse. The observed multi-node trend is discussed conservatively in Section 4; no network or phase-level profile is available in the current result set.

2.3 Alternatives that were rejected

The first CPU-parallel version exposed four concrete implementation problems:

- **False sharing:** adjacent, unaligned per-thread telemetry structures generated unnecessary cache-coherence traffic.
- **Repeated fork–join:** opening and closing an OpenMP region at every epoch added recurring thread-management overhead.
- **Serial deposits:** pheromone deposits executed inside a single-thread section and serialized part of every epoch.
- **Fine-grained MPI synchronization:** one nonblocking reduction per matrix row produced a large number of small collectives.

These problems motivated the organization retained in the submitted version: one persistent OpenMP region, aligned thread-private workspaces, atomic parallel deposits, and sparse delta exchange through `MPI_Iallgatherv`. The changes remove the original serial and per-row synchronization structures, but do not make coordination free. Atomic contention, phase barriers, replicated dense-matrix work, and collective synchronization remain relevant when the ant population does not provide enough independent work per worker.

Further alternatives exposed the following limitations:

- **Scheduling overhead and imbalance:** fine-grained dynamic scheduling repeatedly accesses the OpenMP work queue, whereas larger guided chunks can leave a long tail because ants construct routes of different lengths.
- **Insufficient task granularity:** a fixed population may expose fewer independent ants than threads. Once every ant has been assigned, extra workers can only assist the dense matrix phases; this limitation remains visible in the strong-scaling campaign.
- **Dense-matrix pressure:** double precision enlarged the working set and memory traffic. Single precision reduces this cost but preserves quadratic storage.
- **Irregular fallback:** if the candidate list has no feasible unvisited customer, selection falls back to an $O(n)$ scan. Two-level candidate lists, hierarchical visited masks, and cache-adaptive candidate widths reduce exhausted-block work, but cannot remove the global fallback without changing the search rule.
- **Poor prefetch and SIMD fit:** the visited-dependent access pattern frustrates manual prefetching, while direct SIMD requires masked filtering, reduction, and probabilistic selection and can evaluate entries that are immediately discarded.
- **Fine-grained intra-ant coordination:** per-step barriers repeatedly synchronized workers; atomic spinning replaced barrier calls with cache-line bouncing; and condition-variable wakeups introduced operating-system latency comparable to the small scan being parallelized.

The final one-ant-per-worker mapping therefore synchronizes only at phase boundaries. Hierarchical masks and adaptive candidate data are retained, while the rejected implementation history is reported qualitatively and no cross-version speedup is reused.

2.4 Correctness boundary

The current validation reconstructs every saved route and checks customer coverage and capacity. All available OpenMP–MPI routes are feasible. For several multi-rank runs, however, the scalar `best_cost` does not match the cost of the route printed by rank zero. This does not change the fixed-epoch wall time, which is retained for scaling, but it prevents using those cost values as multi-rank quality evidence. Resolving the ownership and transfer of the globally best route is therefore a necessary correctness improvement beyond the timing study.

3 CUDA Implementation

The CUDA backend changes both representation and work mapping to fit a single GPU. The host parser transfers an $O(n)$ array of `float2` coordinates, and device code computes Euclidean distances on demand. This avoids the additional dense distance matrix used by the CPU implementations. The pheromone matrix remains dense but is log-quantized to one byte per edge, reducing its storage from four or eight bytes per entry to one.

3.1 Warp-per-ant construction

One warp cooperates on each ant. Its lanes score candidate customers, use warp reductions and prefix operations for probabilistic selection, and fall back to a global scan when the candidate list is exhausted. Visited state is represented by hierarchical bit masks. Route construction and cost evaluation remain on the device, while only summaries and a selected final solution need to cross the host–device boundary.

The mapping trades additional lane cooperation for regular GPU execution. Candidate arrays are contiguous, and warp shuffles exchange values without block-wide barriers. Evaporation applies saturated subtraction to the quantized pheromone matrix. Deposit kernels update packed 8-bit values through a 32-bit `atomicCAS` loop, since native byte-wide atomics are unavailable for this operation.

3.2 Rejected CUDA designs

The first GPU design exposed four main problems:

- **Warp divergence:** assigning one ant to each CUDA thread made neighboring lanes follow different paths because of stochastic decisions, unequal route lengths, and occasional global fallback scans.
- **Non-coalesced access:** separate contiguous route and visited buffers for each ant caused neighboring threads to access distant addresses.
- **Quadratic memory pressure:** dense floating-point distance and pheromone matrices imposed $O(n^2)$ capacity and bandwidth costs that limited the instance size.
- **Atomic contention:** allowing every ant to scatter pheromone deposits concentrated atomic updates on popular edges.

The retained design addresses these problems with warp-per-ant cooperation, step-interleaved route storage, coordinate-based distance evaluation, one-byte pheromone values, and deposits restricted to the selected iteration and global best solutions. These choices trade the rejected divergence and memory pressure for repeated distance arithmetic, quantization error, packed-word atomic updates, and fewer independent ants per fixed number of GPU threads.

3.3 Capacity and trade-offs

For $n = 64{,}000$, a byte pheromone matrix contains about 4.1 billion entries, whereas a float matrix would require about 16.4 GB. Coordinate-based distance evaluation also removes a second quadratic array. The design therefore prioritizes capacity as well as throughput, at the cost of quantization error and repeated distance arithmetic.

The evaluation reports only the available end-to-end measurements and explicitly distinguishes stagnation stops from the 300 s limit. No occupancy or memory-bandwidth claim is made without compatible profiling evidence.

4 Experimental Evaluation

4.1 Method and validation

Each experimental point is the arithmetic mean of three seeds, and error bars are the sample standard deviation. Median, extrema, coefficient of variation (CV), and per-run validation outcomes are also considered.

Every current CSV row has positive finite timing and a matching solution file. Reconstructed routes visit every customer exactly once and satisfy capacity. Seven multi-rank outputs have a feasible printed route whose recomputed cost differs from the printed scalar best cost; their timings are retained, but their costs are excluded.

4.2 Strong scaling

The strong experiment fixes $n = 32{,}000$, $K = 32$, the total population at $m = 16$, and the work at 100 epochs; stagnation is disabled. OpenMP measurements use one MPI rank and up to 32 threads, while the distributed measurement uses four ranks with 32 threads per rank. We define $p = \text{ranks} \times \text{threads}$, $S_p = T_1/T_p$, and $E_p = S_p/p$.

Mean OpenMP runtime falls from 165.98 ± 2.11 s at one thread to 60.09 ± 8.39 s at 32 threads. The corresponding speedup is only 2.76 and efficiency is 8.63%; variability also grows to a CV of 13.96%. The fixed population explains an important part of this behavior: the ant-construction loop exposes only 16 independent ants per epoch, so at 32 threads at least half of the workers cannot receive an ant-construction task. Additional threads can still accelerate dense evaporation and other parallel loops, but they do not increase concurrency in the principal coarse-grained phase. End-to-end initialization, phase barriers, atomic deposits, and dense-matrix traffic further bound the attainable speedup. The

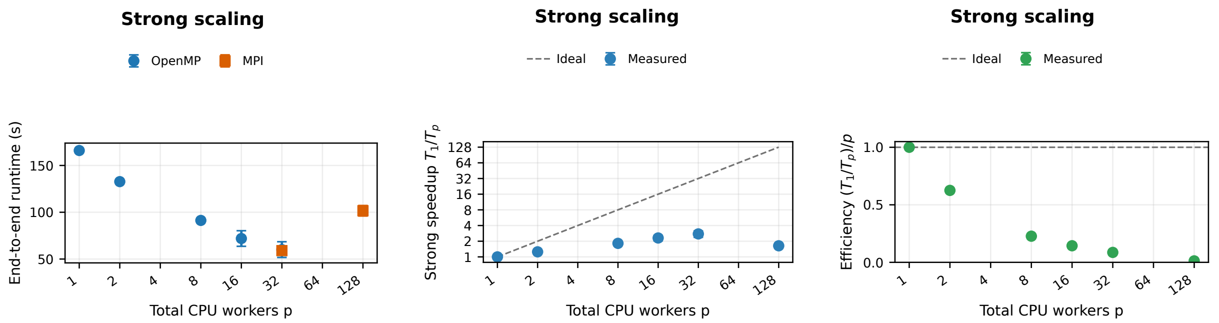

Figure 1: Strong-scaling runtime, speedup, and efficiency as three distinct plots. Points show means of three runs; runtime bars are sample SD and ratio bars use first-order SD propagation. The combined curves use OpenMP within one node and the four-rank MPI result at 128 workers.

measurements therefore show insufficient task granularity for this strong-scaling configuration, while profiling would be needed to quantify the contribution of each remaining phase.

The independent 32-worker MPI baseline is 59.00 ± 3.77 s, compatible in scale with the OpenMP 32-thread measurement. Four ranks require 101.60 ± 5.60 s. Relative to the 32-worker MPI point this is a speedup of 0.58 and an efficiency of 14.5% after normalizing by the fourfold resource increase. Relative to the one-thread combined baseline, the 128-worker point has speedup 1.63 and efficiency 1.28%. Here the 16 ants are divided among four ranks, leaving only four local ants for 32 OpenMP threads on each rank during construction. At the same time, every rank stores and evaporates its dense pheromone replica and participates in collective coordination. The added resources thus increase replicated and coordination work without adding ant work, which accounts for the direction of the observed slowdown; a phase profile would still be required to apportion its magnitude.

4.3 Weak scaling

The weak campaign fixes $n = 8{,}000$ and 100 epochs while maintaining 32 ants per total CPU worker. OpenMP covers 1–32 workers on one rank; MPI extends the curve to 64 and 128 workers with 32 threads per rank. The population therefore grows from 32 to 4096 and the workload-per-worker relation is constant. I report $E_p^{weak} = T_1/T_p$ and use the OpenMP point at 32 workers in the combined curve. The independent one-rank MPI measurement at the same point is retained as a consistency check.

Runtime is 29.46 ± 0.10 s at one worker, remains between 25.47 and 30.43 s through 32 OpenMP workers, falls to 22.05 ± 1.65 s at 64, and rises to 33.87 ± 1.70 s at 128. Weak efficiency is 1.16 at 32, 1.34 at 64, and 0.87 at 128. The 32-worker MPI control is 25.42 ± 0.65 s versus 25.47 ± 0.44 s for OpenMP, supporting continuity at the backend transition. Unlike strong scaling, this campaign supplies 32 ants per worker, so the coarse-grained loop remains populated as resources grow. This directly explains why weak efficiency is much better than strong efficiency. Ratios above one are consistent with amortization of fixed phase costs, but cache, scheduling, and hardware effects cannot be separated without profiling.

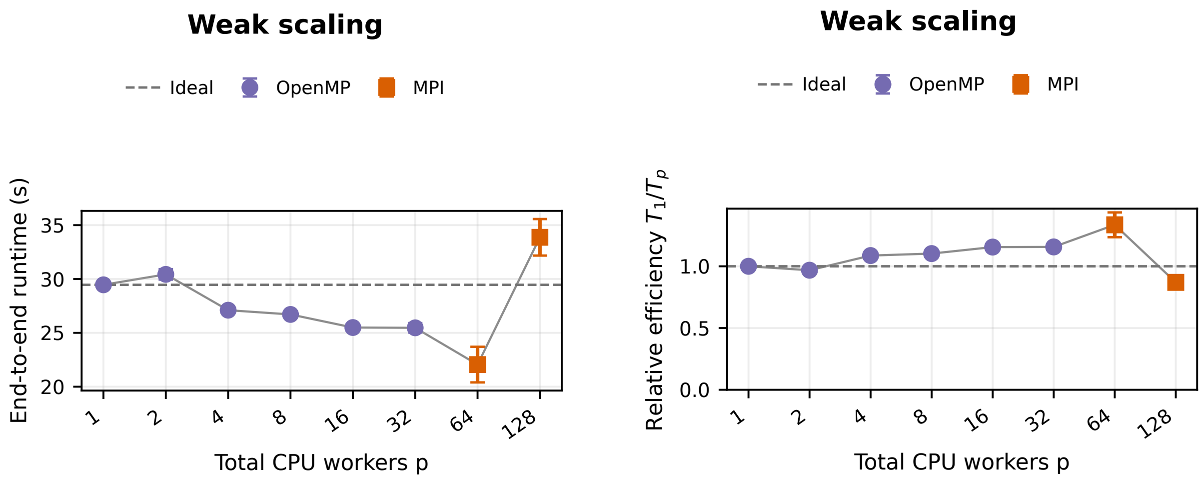

Figure 2: Weak-scaling runtime and efficiency as distinct plots. OpenMP supplies 1–32 workers and MPI supplies 64–128. Points are three-run means with sample-SD bars; efficiency is T_1/T_p.

4.4 Sequential and CUDA size sweep

The size sweep covers sequential runs through $n =$ 32,000 and CUDA through $n = 64{,}000$. It is not a like-for-like speedup experiment: CUDA uses $m =$ 256, while the sequential population varies from 32 to 4, and both stop on stagnation or at 300 solver seconds. Sequential runs hit the time limit from $n =$ 2,000 onward; CUDA hits it at $n = 64{,}000$. External times can exceed 300 s because initialization and finalization are outside the solver timer.

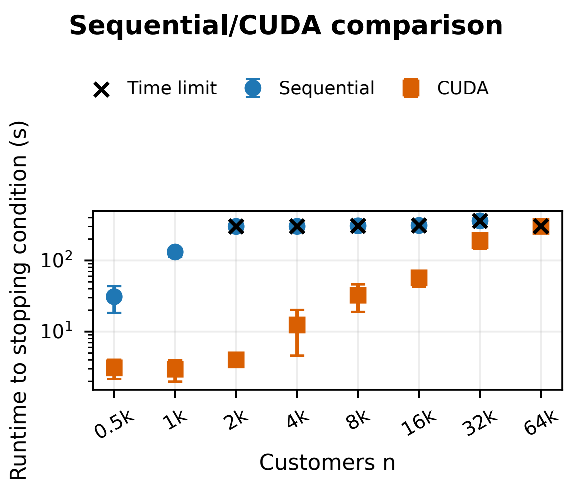

Figure 3: Runtime in the descriptive size sweep. Points are three-run means with sample-SD bars; crosses mark time-limited groups.

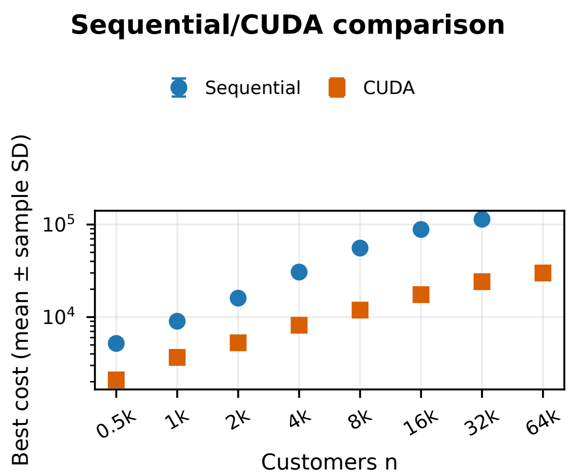

Figure 4: Cost in the descriptive size sweep. Points are three-run means with sample-SD bars. Different populations and stopping epochs preclude interpreting the comparison as a backend speedup.

CUDA reaches $n = 64{,}000$ within the available outputs, while the dense sequential backend reaches $n = 32{,}000$ with about 26.7 GiB RSS. Runtime variability is substantial for several stagnation-terminated CUDA points (CV 62.9% at $n = 4{,}000$ and 41.8% at $n = 8{,}000$), so a single best run would be misleading. Although CUDA reports lower runtime and cost on all shared sizes, the unequal search effort means that these observations cannot establish an implementation speedup or equal-budget quality advantage.

5 Conclusions

The reliable current evidence supports three limited conclusions. First, OpenMP reduces the 100-epoch runtime on the $n = 32{,}000$ instance, but a population of only 16 ants leaves the 32-thread configuration underutilized and limits speedup to 2.76×. Second, splitting the same fixed population over four MPI ranks increases replicated and collective work while leaving only four ants per rank, producing a multi-node slowdown. In contrast, the weak campaign keeps enough independent ants per worker and preserves 87% efficiency at 128. Third, the CUDA representation handles the largest available instance, but the present size sweep is not controlled well enough for a cross-backend speedup claim.

These results characterize the submitted configuration rather than a general limit of OpenMP or MPI. Its strong-scaling resource count grows beyond the available ant-level parallelism, whereas the weak campaign grows useful work with the worker count. Phase profiling would be necessary for a more detailed attribution. The inconsistent transfer of the globally best MPI route also remains a correctness limitation for quality comparisons, although its fixed-work timings are retained.

References

[1] Wikipedia, *Vehicle routing problem*: `https://en.wikipedia.org/wiki/Vehicle_routing_problem`

[2] Wikipedia, *Ant colony optimization algorithms*: `https://en.wikipedia.org/wiki/Ant_colony_optimization_algorithms`